

Big Data Analytics Technology
MSc in Data Science, University of Coventry (UK)

Coursework

Student Name	Student ID
M D R D Goonatilaka	COMScDS252P-012

Contents

Executive summary	3
Task 1: Distributed Time-series Data Management using InfluxDB	4
Task 2: Real-Time Stream Ingestion and processing with Apache Kafka and Apache Flink ...	8
Task 3: Scalable Data Analytics with Apache Spark.....	13
Task 4: Graph Database Engineering using Neo4j.....	18
Task 5: Theoretical Foundations and Future Engineering Trends.....	22
Reference List.....	26

Executive summary

This report is split into five tasks: the first task is about distributed time-series using InfluxDB, the second task is about Real-Time stream Ingestion & Processing with Apache Kafka and Apache Flink, task three is about scalable Data Analytics with Apache Spark, task four is about Graph Database Engineering using Neo4j and task five about Theoretical Foundations and Future Engineering Trends. The result was generated using Vs code platform and docker desktop to access platforms like spark, InfluxDB, Kafka, Flink and Neo4j.

Please note the link below contains the python and docker compose code (covering all four task):

GitHub link: <https://github.com/247953/Big-Data-Analytics-CW-COMScDS252P-012.git>

Also, note task 4 datasets are linked to the following one drive link due to large file size extending GitHub Repository requirements:

https://nibm-my.sharepoint.com/:f/g/personal/comscds252p-012_student_nibm_lk/IgDUgYgy13ZDRrKRmyZ3WphuAQryg1XytEwuT5T7kGTawH8?e=Eq6a4f

Task 1: Distributed Time-series Data Management using InfluxDB

1. Overview

This project provisions a time-series analytics pipeline using Docker, Influx DB v2.7, Python and Influx script language. The primary objective is to ingest, model and analyse 96,453 historical climate observations from Szeged, Hungary (2006-2016).

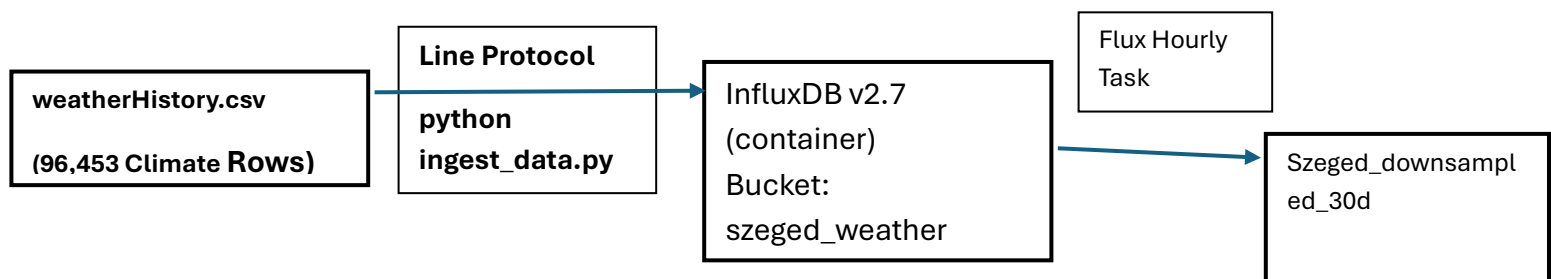
The pipeline supports high-throughput ingestion, multi-decadal time series windowing, dynamic statistical anomaly isolation ($\pm 2\sigma$), and continuous automated down sampling with clear 30-day retention policy.

Target Environment: InfluxDB v2.7 (Docker Container)

Dataset: [Weather in Szeged 2006-2016 | Kaggle](#)

Component application summary

- Database Platform: InfluxDB v2.7 running inside a docker container configured via docker-compose.yml file through vs code.
- Data Ingestion: Python script utilizing pandas and the InfluxDB-client library to parse multi-scenario climate metrics and stream InfluxDB Line Protocol points.



2. InfluxDB setup and Continuous automation

2.1 Bucket Retention Configuration

The auxiliary bucket was named `szeged_downsampled_30d` (Bucket ID: 96b4c33e840454d3) which was illustrated on the Influx DB Web UI with a clear 30-day retention period thereby enabling to continuously prune historical down sampled metrics and optimize system storage footprint.

2.2 Continuous Down sampling task

When looking to apply an automated InfluxDB Task named as **downsample_szeged_hourly**, it was configured to execute every 1 hour. In simple terms, the task queries raw climate metrics from

the preceding 2-hour window, is computed an hourly mean using `aggregateWindow ()`, thus relabelling the measurement identifiers and directing the result toward `szeged_downsampled_30d`.

This is demonstrated on the code entered on InfluxDB UI window:

```
option task = {name: "downsample_szeged_hourly", every: 1h}

from(bucket: "szeged_weather")
  |> range(start: -2h)
  |> filter(fn: (r) => r["_measurement"] == "weather_metrics")
  |> aggregateWindow(every: 1h, fn: mean, createEmpty: false)
  |> set(key: "_measurement", value: "weather_metrics_hourly")
  |> to(bucket: "szeged_downsampled_30d", org: "szeged_org")
```

3. Data queries and Flux queries

Query 1: Sliding window aggregation

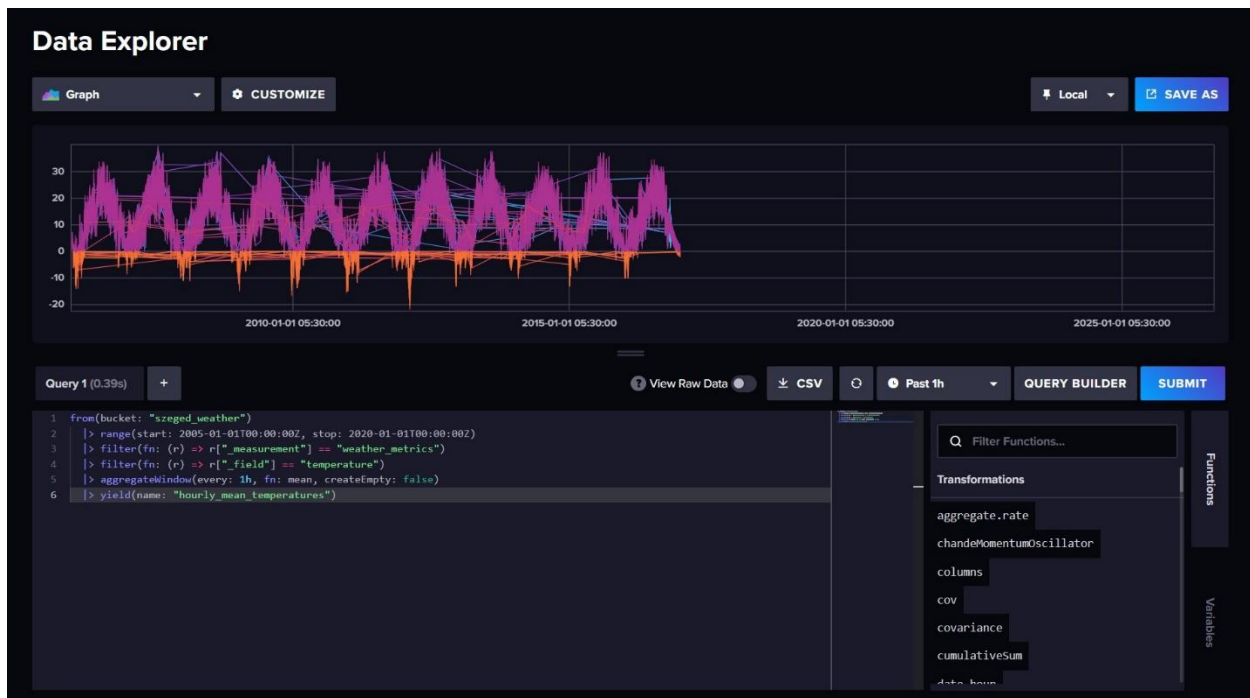
Groups multi-year metrics into hourly interval window using *aggregateWindow ()* to construct trend visualizations while suppressing short-term high frequency noise.

Query 2: Dynamic Anomaly Detection ($\pm 2\sigma$)

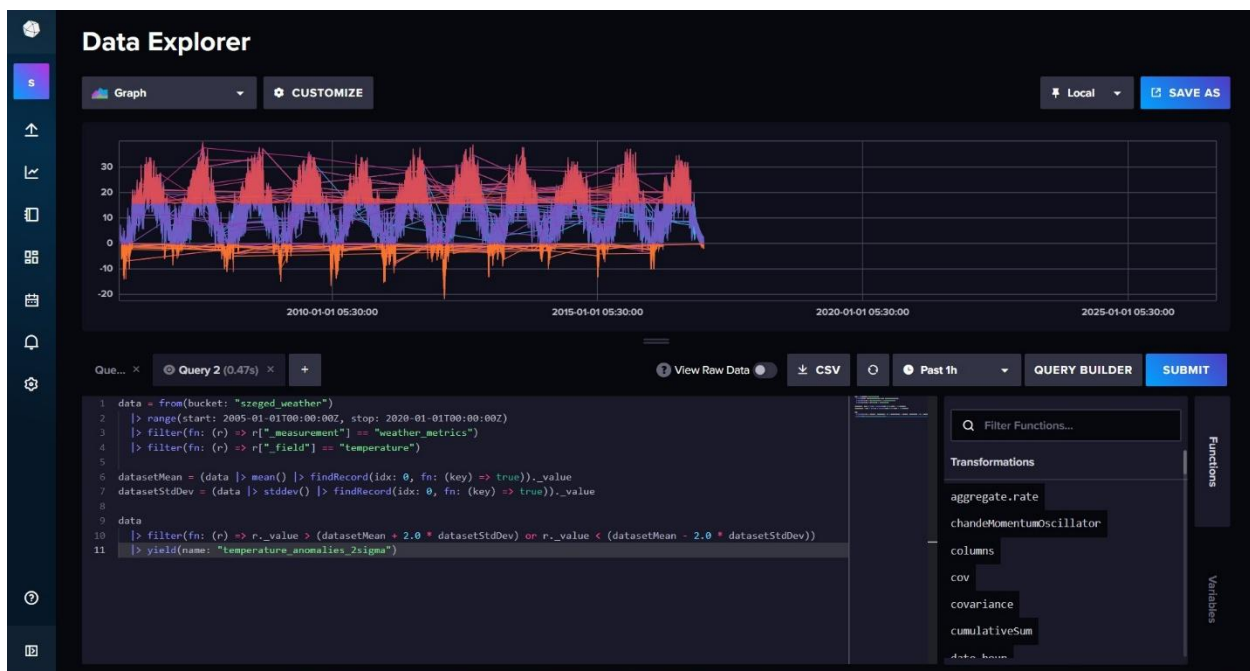
To isolate statistically significant climate anomalies, a two-pass Flux script dynamically compute the global mean (μ) and standard deviation (σ) across the target field. Also, Data points exceeding two standard deviations ($|x-\mu| > 2\sigma$) were extracted as isolated anomalies.

4. The Verification results

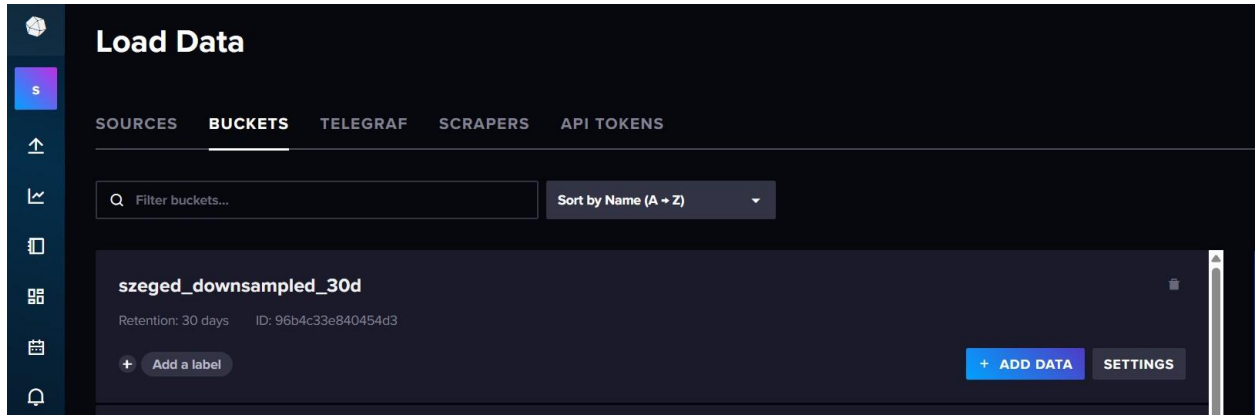
1. 01 query sliding window: The screenshot below showcases the full InfluxDB UI Data Explorer view displaying Query 1 sliding window hourly aggregation line graph over the extended time horizon.



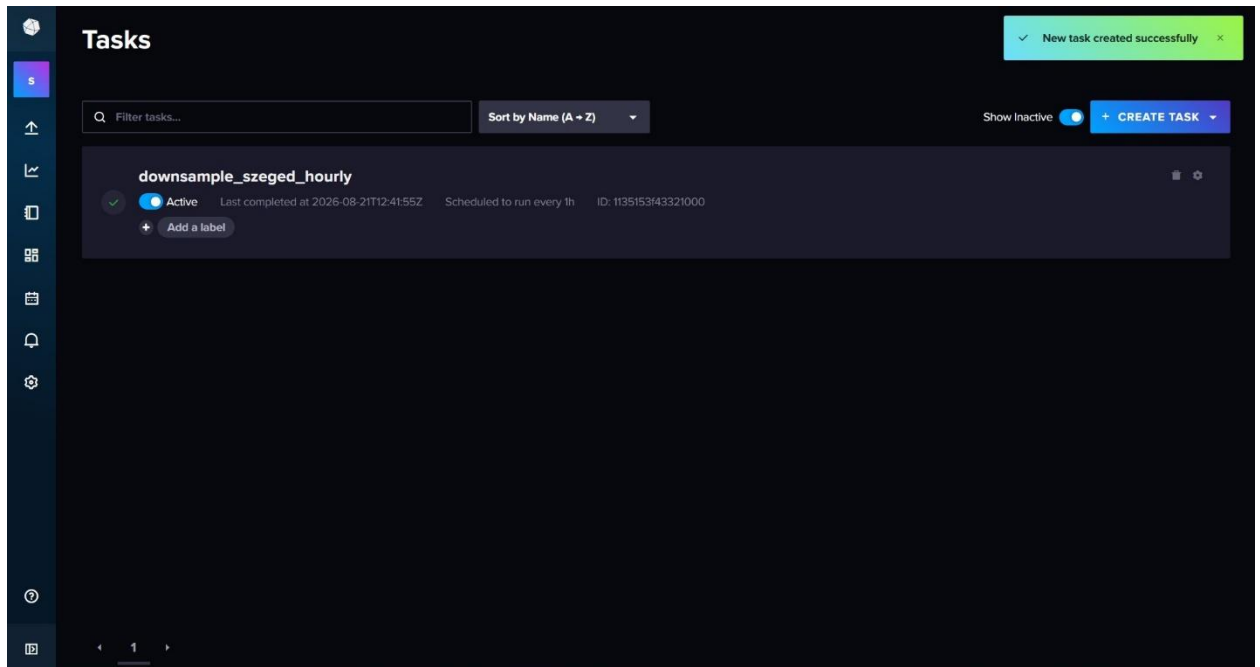
- 02 Query Anomaly detection: Data explorer table/graph view isolating extreme temperature events outside 2σ .



3. Bucket retention 30 days: InfluxDB settings view displaying the active `szeged_downsampled_30d` bucket with 30-day retention tag (displayed on the screenshot below).



4. Active down sampling task: InfluxDB tasks manager view verifying `downsample_szeged_hourly` scheduled every 1 hour with an active status toggle and green execution benchmark displayed on the screenshot below.



Task 2: Real-Time Stream Ingestion and processing with Apache Kafka and Apache Flink

1. Executive Summary

This project demonstrates an end-to-end real-time stream processing pipeline built using Apache Kafka, Apache Flink (PyFlink) and Docker. To carry out this analysis, live traffic data was ingested from the City of Austin Open Data API, streamed through a multi-partitioned Kafka topic and processed using Flink's event-time windowing to aggregate vehicle counts per sensor point.

Link: <https://data.austintexas.gov/Transportation-and-Mobility/Camera-Traffic-Counts/sh59-i6y9>

2. System Architecture and Component

- Data Producer (producer.py)

Python was used to read the periodic poll the Austin Traffic Telemetry endpoint (<https://data.austintexas.gov/resource/sh59-i6y9.json>). Following establishing the API endpoint access, formatted payloads with dynamic camera IDs (camera_id), vehicle counts (volume), locations and live epoch timestamps in milliseconds (timestamp_ms). Also, this involves serializing events as JSON and publishes them to Kafka with camera_id as the record key to preserve partitioning logic.

- Message Broker (Apache Kafka)
 - Topic Name: traffic-telemetry
 - Broker Address: 127.0.0.1:9094
 - Partitioning strategy: The key-based partitioning uses camera_id to ensure ordered message is delivery as per tracking point.
- Stream engine (PyFlink):
 - Watermark strategy: This is to monitor event-time temporal assignment (TIMESTAMP_LTZ)
 - Windowing: Tumbling Windows
 - -Aggregation: This is grouped by camera_id and location, to calculate total vehicle volume and record event counts per window.

3. Pipeline Implementation and verification

3.1 Producer step up and data ingestion

In the image shown below, it shows a custom python script which is used to fetch traffic records and streams telemetry events directly to Kafka (Screenshot 1).


```
# key ingestion and serialization
payload = {
    "camera_id": camera_id,
    "timestamp_ms": timestamp_ms,
    "volume": volume,
    "location": location
}

producer.send(TOPIC_NAME, key=camera_id, value=payload)
print(f"[->] Kafka Event: Camera={camera_id} | Vol={volume} | Time={timestamp_ms}")
```

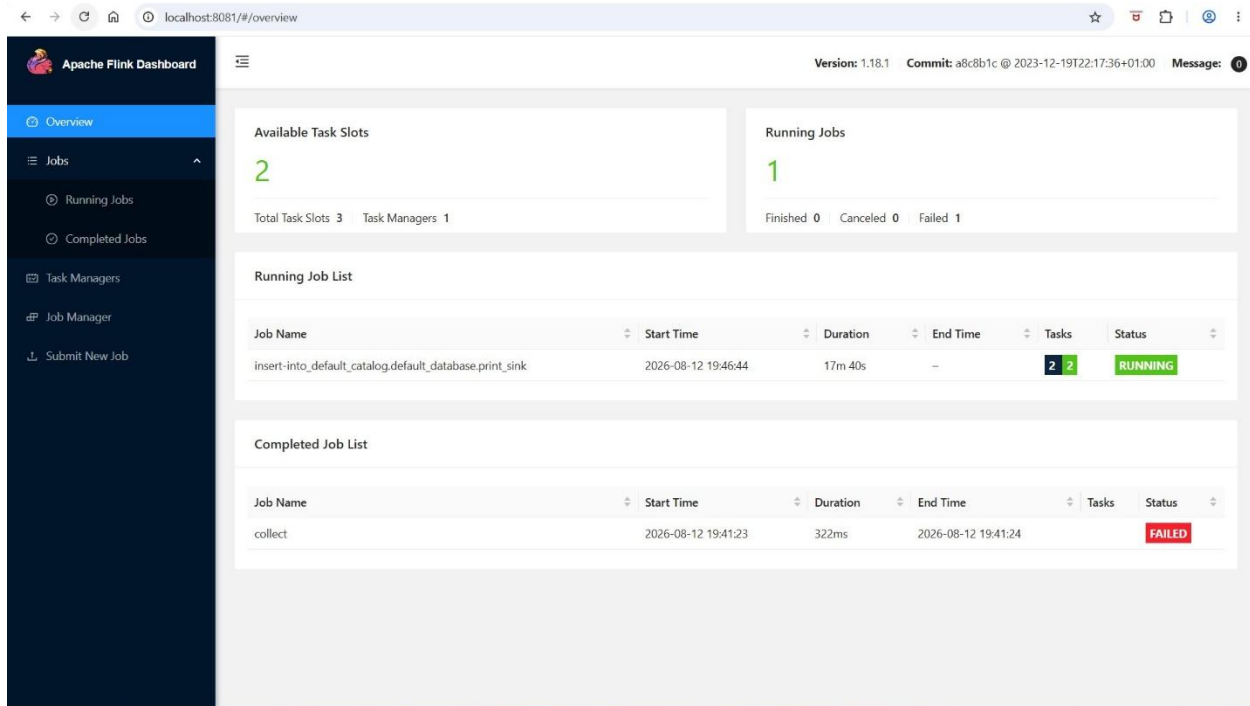
Screenshot 1: custom python script to fetch traffic record

```
PS C:\Users\mdr\OneDrive\Task 2> python producer.py
[->] Kafka Event: Camera=CAM_228 | Vol=3 | Time=1786546468948
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546469951
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546470953
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546471960
[->] Kafka Event: Camera=CAM_228 | Vol=30 | Time=1786546472962
[->] Kafka Event: Camera=CAM_228 | Vol=2 | Time=1786546473963
[->] Kafka Event: Camera=CAM_228 | Vol=21 | Time=1786546474965
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546475966
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546476967
[->] Kafka Event: Camera=CAM_228 | Vol=4 | Time=1786546477969
[->] Kafka Event: Camera=CAM_228 | Vol=6 | Time=1786546478972
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546479981
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546480982
[->] Kafka Event: Camera=CAM_228 | Vol=19 | Time=1786546481984
[->] Kafka Event: Camera=CAM_228 | Vol=2 | Time=1786546482985
[->] Kafka Event: Camera=CAM_228 | Vol=2 | Time=1786546483986
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546484988
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546485989
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546486991
[->] Kafka Event: Camera=CAM_228 | Vol=11 | Time=1786546487996
[->] Kafka Event: Camera=CAM_228 | Vol=2 | Time=1786546488998
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546490001
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546491005
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546492010
[->] Kafka Event: Camera=CAM_228 | Vol=16 | Time=1786546493011
[->] Kafka Event: Camera=CAM_228 | Vol=7 | Time=1786546494013
[->] Kafka Event: Camera=CAM_228 | Vol=2 | Time=1786546495016
[->] Kafka Event: Camera=CAM_228 | Vol=5 | Time=1786546496019
[->] Kafka Event: Camera=CAM_228 | Vol=4 | Time=1786546497019
[->] Kafka Event: Camera=CAM_228 | Vol=2 | Time=1786546498023
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546499030
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546500036
[->] Kafka Event: Camera=CAM_228 | Vol=3 | Time=1786546501038
[->] Kafka Event: Camera=CAM_228 | Vol=5 | Time=1786546502040
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546503040
[->] Kafka Event: Camera=CAM_228 | Vol=3 | Time=1786546504042
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546505045
[->] Kafka Event: Camera=CAM_228 | Vol=5 | Time=1786546506046
[*] Fetched 50 records. Sending to Kafka...
[->] Kafka Event: Camera=CAM_228 | Vol=1 | Time=1786546515681
```

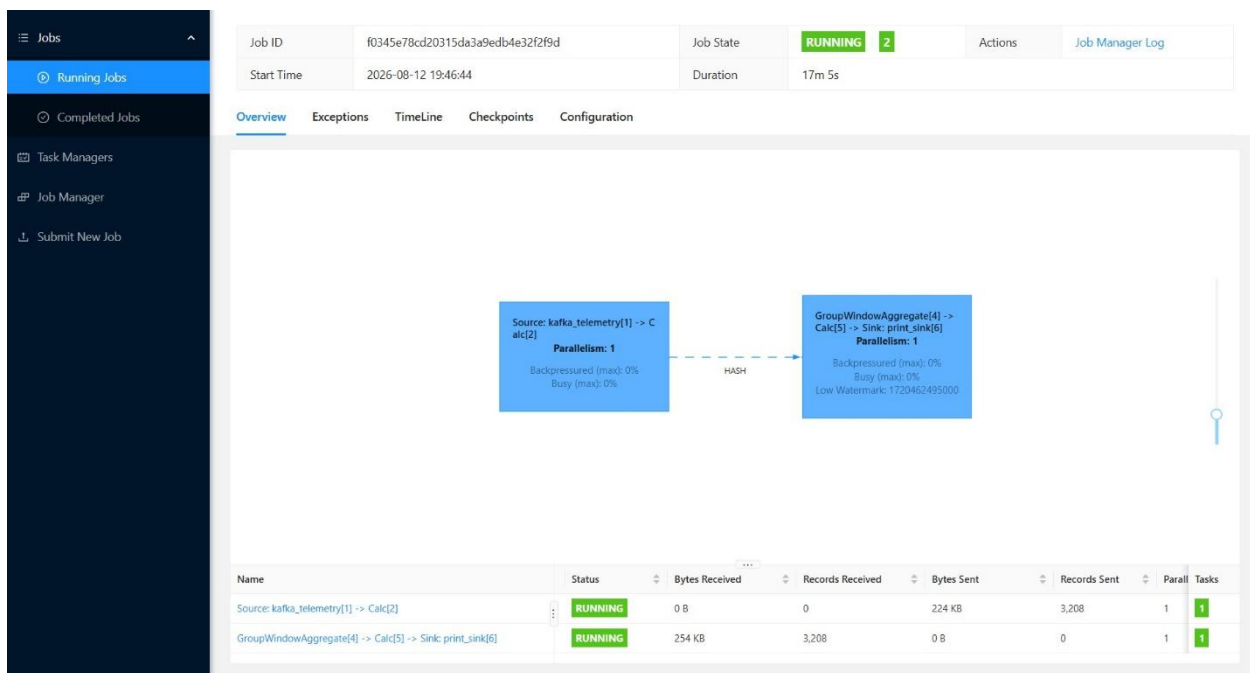
In screenshot 2 above, displays the Producer Terminal Logs showing the output running of the python producer.py file. Moreover, the screenshot showcases the successful REST Get fetches (50 records) and live event emissions.

3.2 Flink Topology and cluster execution

The Flink cluster consumes events from the traffic-telemetry topic and passes through the defined execution graph.



Screenshot 3: Flink web Dashboard-DAG and overview



Screenshot 4: Flink Job execution and Graph (DAG)

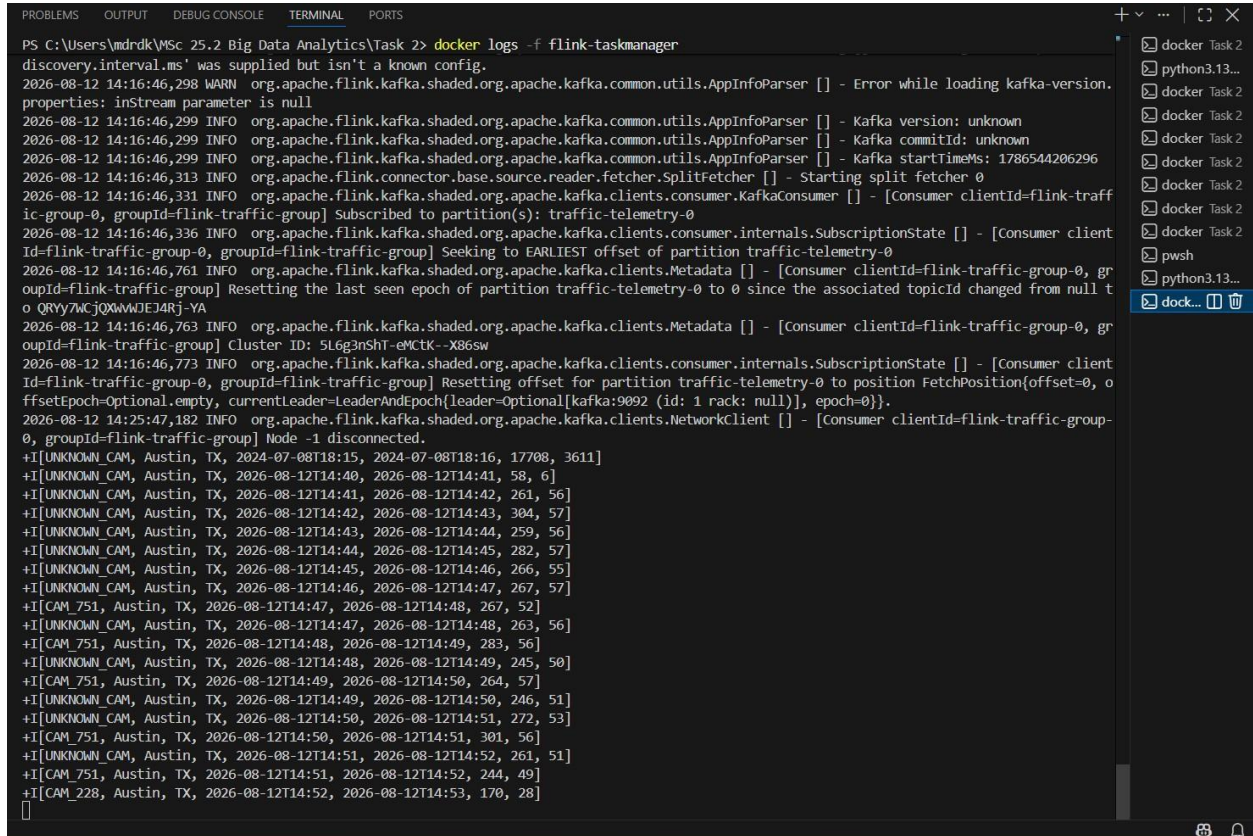
Flink Dashboard as shown by the screenshot 3 above displays the use of localhost:8081, showing 1 active running job (traffic-telemetry job) with the execution topology flowing from Kafka_telemetry to Group Window Aggregate to print_sink. Furthermore, this aspect is

demonstrated by screenshot 4 along with the DAG graph of the entire Flink and Kafka implementation.

3.3 Event-time Window Aggregate Results

As records stream in, Flink's event-time watermarks advance continues, closing window boundaries and releasing window aggregates to the Task Manager Standard output.

Screenshot 5 shows below show the task manger output.



```
PS C:\Users\mdrdk\WS\Big Data Analytics\Task 2> docker logs -f flink-taskmanager
discovery.interval.ms' was supplied but isn't a known config.
2026-08-12 14:16:46,298 WARN org.apache.flink.kafka.shaded.org.apache.kafka.common.utils.AppInfoParser [] - Error while loading kafka-version.
properties: inStream parameter is null
2026-08-12 14:16:46,299 INFO org.apache.flink.kafka.shaded.org.apache.kafka.common.utils.AppInfoParser [] - Kafka version: unknown
2026-08-12 14:16:46,299 INFO org.apache.flink.kafka.shaded.org.apache.kafka.common.utils.AppInfoParser [] - Kafka commitId: unknown
2026-08-12 14:16:46,299 INFO org.apache.flink.kafka.shaded.org.apache.kafka.common.utils.AppInfoParser [] - Kafka startTimeMs: 1786544206296
2026-08-12 14:16:46,313 INFO org.apache.flink.connector.base.source.reader.fetcher.SplitFetcher [] - Starting split fetcher 0
2026-08-12 14:16:46,331 INFO org.apache.flink.kafka.shaded.org.apache.kafka.clients.consumer.KafkaConsumer [] - [Consumer clientId=flink-traff
ic-group-0, groupId=flink-traffic-group] Subscribed to partition(s): traffic-telemetry-0
2026-08-12 14:16:46,336 INFO org.apache.flink.kafka.shaded.org.apache.kafka.clients.consumer.internals.SubscriptionState [] - [Consumer client
Id=flink-traffic-group-0, groupId=flink-traffic-group] Seeking to EARLIEST offset of partition traffic-telemetry-0
2026-08-12 14:16:46,761 INFO org.apache.flink.kafka.shaded.org.apache.kafka.clients.Metadata [] - [Consumer clientId=flink-traffic-group-0, gr
oupId=flink-traffic-group] Resetting the last seen epoch of partition traffic-telemetry-0 to 0 since the associated topicId changed from null t
o 0RYY7WCJQXW4DEJ4Rj-YA
2026-08-12 14:16:46,763 INFO org.apache.flink.kafka.shaded.org.apache.kafka.clients.Metadata [] - [consumer clientId=flink-traffic-group-0, gr
oupId=flink-traffic-group] Cluster ID: 5L6g3nShT-eMCKK-X86sw
2026-08-12 14:16:46,773 INFO org.apache.flink.kafka.shaded.org.apache.kafka.clients.consumer.internals.SubscriptionState [] - [Consumer client
Id=flink-traffic-group-0, groupId=flink-traffic-group] Resetting offset for partition traffic-telemetry-0 to position FetchPosition{offset=0, o
ffsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch{leader=Optional[kafka:9092 (id: 1 rack: null)], epoch=0}}.
2026-08-12 14:25:47,182 INFO org.apache.flink.kafka.shaded.org.apache.kafka.clients.NetworkClient [] - [Consumer clientId=flink-traffic-group-
0, groupId=flink-traffic-group] Node -1 disconnected.
+I[UNKNOWN_CAM, Austin, TX, 2024-07-08T18:15, 2024-07-08T18:16, 17708, 3611]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:40, 2026-08-12T14:41, 58, 6]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:41, 2026-08-12T14:42, 261, 56]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:42, 2026-08-12T14:43, 304, 57]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:43, 2026-08-12T14:44, 259, 56]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:44, 2026-08-12T14:45, 282, 57]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:45, 2026-08-12T14:46, 266, 55]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:46, 2026-08-12T14:47, 267, 57]
+I[CAM_751, Austin, TX, 2026-08-12T14:47, 2026-08-12T14:48, 267, 52]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:47, 2026-08-12T14:48, 263, 56]
+I[CAM_751, Austin, TX, 2026-08-12T14:48, 2026-08-12T14:49, 283, 56]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:48, 2026-08-12T14:49, 245, 50]
+I[CAM_751, Austin, TX, 2026-08-12T14:49, 2026-08-12T14:50, 264, 57]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:49, 2026-08-12T14:50, 246, 51]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:50, 2026-08-12T14:51, 272, 53]
+I[CAM_751, Austin, TX, 2026-08-12T14:50, 2026-08-12T14:51, 301, 56]
+I[UNKNOWN_CAM, Austin, TX, 2026-08-12T14:51, 2026-08-12T14:52, 261, 51]
+I[CAM_751, Austin, TX, 2026-08-12T14:51, 2026-08-12T14:52, 244, 49]
+I[CAM_228, Austin, TX, 2026-08-12T14:52, 2026-08-12T14:53, 170, 28]
```

It is important to note screenshot 3 showcases the Terminal output from docker logs -f flink-taskmanager, which demonstrates overall window complete prints (identified via +I [...]) grouped by distinct camera keys.

4. Technical Challenges and Mitigation

Static Historical API Timestamps:

- Issue: Socrata API records contained fixed past dates (2024-07-08), which prevented Flink event-time watermarks from advancing.
- Resolution: Injected current epoch millisecond timestamps (int (time.time () * 1000)) at stream time to keep event-time watermarks in sync with real time.

API Response Fallbacks and Unassigned Camera keys:

- Issue: Intermittent API field variations caused camera IDs to default to “UNKNOWN_CAM”, hence combining all telemetry into a single partition key.
- Solution: implemented fallback key resolution evaluating camera_id, detector_id, atd_location_id and dynamic location-hashing (CAM_XXX), to ensure distinct key grouping across sensors.

API Endpoints Returning HTTP 400 Errors:

- Issue: Sending raw POST queries to the v3/view endpoint yielded 400 Bad Request responses due to header/query restrictions.
- Solution: Migrated the ingestion engine to standard SODA REST GET requests using parameter limits and descending data order (\$limit=50&\$order=read_date%20DESC)

5.Conclusion

The pipeline successfully demonstrates an event-driven big data streaming architecture. Applying and utilizing Apache Kafka for message ingestion as well as using Apache Flink for event-time tumbling window aggregations, real-time traffic volume data is processed clearly with watermarking and window triggering.

Task 3: Scalable Data Analytics with Apache Spark

System configuration and Environment

The graph processing pipeline was executed on a local multi-container Apache Spark 3.5.1 deployment using Docker Compose. The compute cluster is comprised of one Spark Master container and two dedicated Spark worker containers which is designed as follows:

- **Dataset link:** [SNAP: Network datasets: Berkeley-Stanford web graph](#)
- **Master Container (spark-master):** Directed job scheduling and coordinated driver operations. Moreover, it is responsible to exposing the spark into web UI endpoint on port 8080 (cluster management) and port 4040 (application execution metrics).
- **Worker Containers (spark-worker-1, spark-worker-2):** Provisioned with 2 CPU cores and 2.0 GiB RAM each, providing a total cluster compute capacity of 4 virtual cores and 4.0 GiB executor memory (shown on screenshot 1 below).
- **Storage and Volume Mounts:** The Host directories were mapped to /opt/spark/data and /opt/spark/scripts inside the containers to grant persistent access to input datasets and PySpark scripts.

← → ↺ 🔍 localhost:8080/#completed-app ☆ 🗑️ 📄 🔄 ⋮

Spark Master at spark://172.20.0.2:7077

URL: spark://172.20.0.2:7077
Alive Workers: 2
Cores in use: 4 Total, 4 Used
Memory in use: 4.0 GiB Total, 4.0 GiB Used
Resources in use:
Applications: 1 Running, 0 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

▼ Workers (2)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260819062850-172.20.0.3-46249	172.20.0.3:46249	ALIVE	2 (2 Used)	2.0 GiB (2.0 GiB Used)	
worker-20260819062850-172.20.0.4-39889	172.20.0.4:39889	ALIVE	2 (2 Used)	2.0 GiB (2.0 GiB Used)	

▼ Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260819062900-0000	(kill) SNAP Web-Berkstan In-Degree Analysis	4	2.0 GiB		2026/08/19 06:29:00	spark	RUNNING	2.4 min

▼ Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Screenshot1: Spark Master overview cluster

In-degree aggregation Results (web-BerkStan)

The directed web graph dataset (web-BerkStan.txt) was inputted, filtered to strip SNAP metadata lines (#), parsed into explicit (FromNodeId, ToNodeId) edge pairs and cached in memory. An in-degree group-by-aggregation was calculated on ToNodeId to identify dominant hub destination nodes.

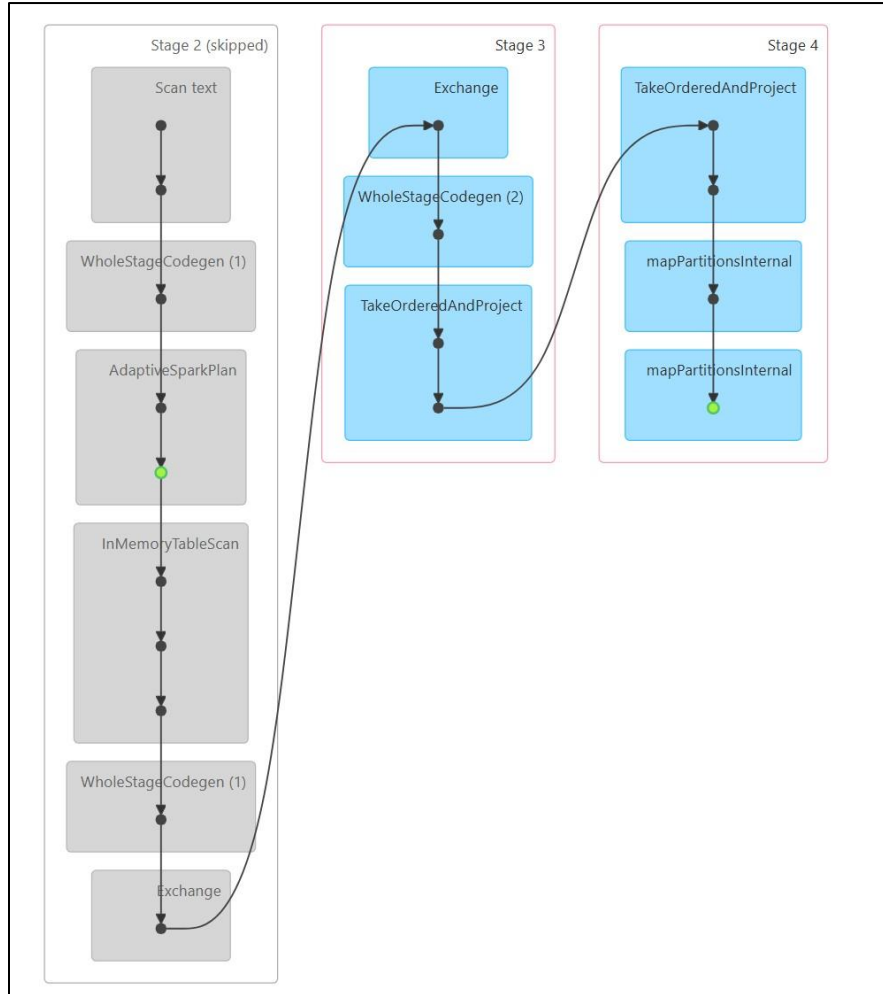
This result in delivering 50 results when entered on Vs code terminal, the image (screenshot 2) below showcases the top 10 dominant destination nodes (in-degree):

```
PS C:\Users\mdrdk\MSc 25.2 Big Data Analytics\Task 3> docker exec -it spark-master /opt/spark/bin/spark-submit --master spark://spark-master:7077 --executor-memory 2G --total-executor-cores 4 /opt/spark/scripts/indegree_analysis.py
```

ToNodeId	in_degree
438238	84208
401873	48205
184094	44290
768	44101
927	44067
184142	44041
184279	44037
184332	44034
33	44032
743	44022

Graph Analysis: Node 438238 acts as the primary central authority within the network graph, receiving nearly double the incoming hyper-links of the second-ranked node (401873). Also, the rapid drop-off incoming link counts from 84208 to ~7500 by rank 50 which demonstrates the classic scale free power law degree distributional characteristics of web topology graphs.

Directed Acyclic Graph (DAG) and Execution Stages



Screenshot 3: Spark DAG Visualization for Job ID 2

In screenshot 3, the image depicts cached memory scan (*InMemoryTableScan*), wide dependency shuffle (Exchange) and the result reduction stages.

- **Skipped Stage Optimisation (stage 2);** This is because the parsed edges Data Frame persisted using the function `edges_df.cache()`, raw text re-parsing (scan text) was skipped during downstream operations, loading pre-parsed edges directly from in-memory cache (shown on screenshot 3 above).
- **Shuffle Exchange (stage 3):** A wide dependency shuffle transformation following Exchange, portioned records across worker nodes using the hash key of `ToNodeId`. *WholestageCodegen (2)* compiled the group by count before executing a partial *TakeOrderedAndProject* operation to keep top local candidates.

- **Final Reduction (stage 4):** partitioned candidate sets were gathered at driver via *mapPartitionInternal* to finalize the global top 50 array output for. show (50).

Distributed Performance Metrics and Workload Balance

Exact performance metrics extracted from stage 3 of the Spark Application UI (<http://localhost:4040>) shown on screenshot 4 below:

Details for Stage 3 (Attempt 0)

Resource Profile Id: 0
 Total Time Across All Tasks: 9 s
 Locality Level Summary: Node local: 200
 Shuffle Read Size / Records: 5.4 MiB / 645473
 Shuffle Write Size / Records: 137.0 KiB / 10000
 Associated Job Ids: 2

▶ DAG Visualization
 ▶ Show Additional Metrics
 ▶ Event Timeline

Summary Metrics for 200 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	8.0 ms	24.0 ms	35.0 ms	52.0 ms	0.4 s
GC Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	34.0 ms
Shuffle Read Size / Records	26.4 KiB / 3083	27.3 KiB / 3184	27.6 KiB / 3222	27.9 KiB / 3269	28.8 KiB / 3378
Shuffle Write Size / Records	639 B / 50	689 B / 50	701 B / 50	716 B / 50	749 B / 50

▶ Aggregated Metrics by Executor

Tasks (200)

Show 20 entries Search:

Screenshot 4: Spark Application UI stage 3 details

Screenshot 4 displays Spark application UI stage 3 details illustrating shuffle read/write data sizes and task duration quantile duration.

Details as follows:

- **Total Task Processing Time:** 9 seconds across all 200 executor tasks
- **Shuffle Data Transfer:**
 - Shuffle Read Size/ Records: 5.4 MiB/ 645,473 records transferred across partitions
 - Shuffle Write Size/ Records: 137.0 KiB/ 10,000 records emitted by mapper tasks
- **Task Duration Quantiles and Skew Analysis:**
 - MIN:8.0 ms
 - 25th percentile: 24.0 ms

- Median (50th percentile): 35.0 ms
 - 75th percentile: 52.0ms
 - Max: 0.4s
-
- **Workload Balance Conclusion:** Across 200 tasks, the median execution time was 35.0 ms with a maximum of 0.4s seconds. This level of low spread highlights that spark's default 200 hash-partitioning strategy has effectively distributed to the highest degree of node aggregation across the 4 executor cores without triggering any potential bottlenecks.

Task 4: Graph Database Engineering using Neo4j

In this report, the focus is to evaluate the structural properties of a directed patent citation network using Neo4j. This is conducted by containerizing instance of Neo4j using Docker Desktop. Containerization provides a deterministic runtime environment, preventing host-level dependency conflicts and to facilitate reproducible query benchmarks.

Source: US Patents from 1963-1999

Link: [SNAP: Network datasets: US Patent citation network](#)

Platforms: Vs code Terminal and Docker

1. Docker Runtime configuration

The container instance was instantiated with named persistent volumes to decouple the database engine from the underlying host file system whilst exposing essential network interfaces:

- Container Name: neo4j-patents
- Container Identifier: b968089bf445
- Docker Image: neo4j: latest
- Port Bindings:
 - HTTP (7474): Access the Neo4j browser web interface
 - Bolt (7687): Handles high-speed database client connection

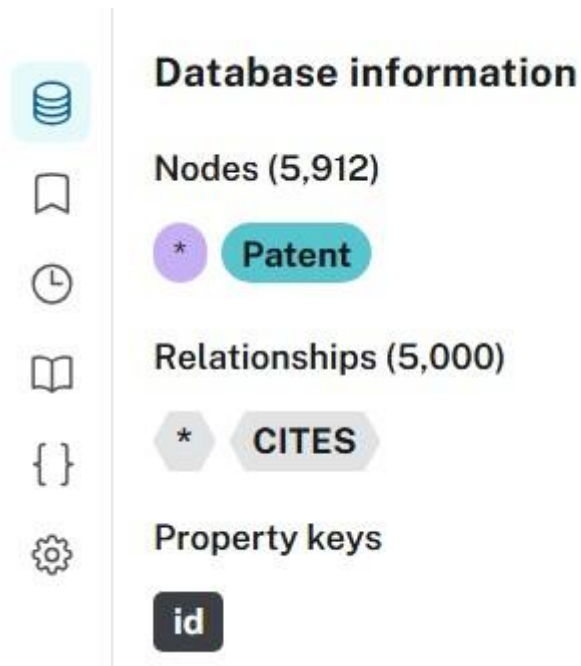
```
PS C:\Users\mdrdk\MSc 25.2 Big Data Analytics\Task 4> docker ps -f "name=neo4j-patents"
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
b968089bf445   neo4j:latest   "tiny -g -- /startup..." 14 minutes ago Up 14 minutes 0.0.0.0:7474->7474/tcp, [::]:7474->7474/tcp, 0.0.0.0:7687->7687/tcp, [::]:7687->7687/tcp neo4j-patents
```

The above image is a screenshot of the Terminal output verifying the active Docker container (neo4j-patents) and port mapping.

2. Database Overview and Graph Structure

The database models a patent network where individual patents are Nodes and the citations between them are Relationships.

- Total Nodes: 5912 (: patent label, with property id)
- Total relationships: 5000 (: CITIES directed edges)



The screenshot above highlights the Neo4j browser sidebar displaying the total database statistics (5912 patent nodes and 5000 CITES relationships).

3. Query Analysis and Graph

3.1 Shortest path Traversal

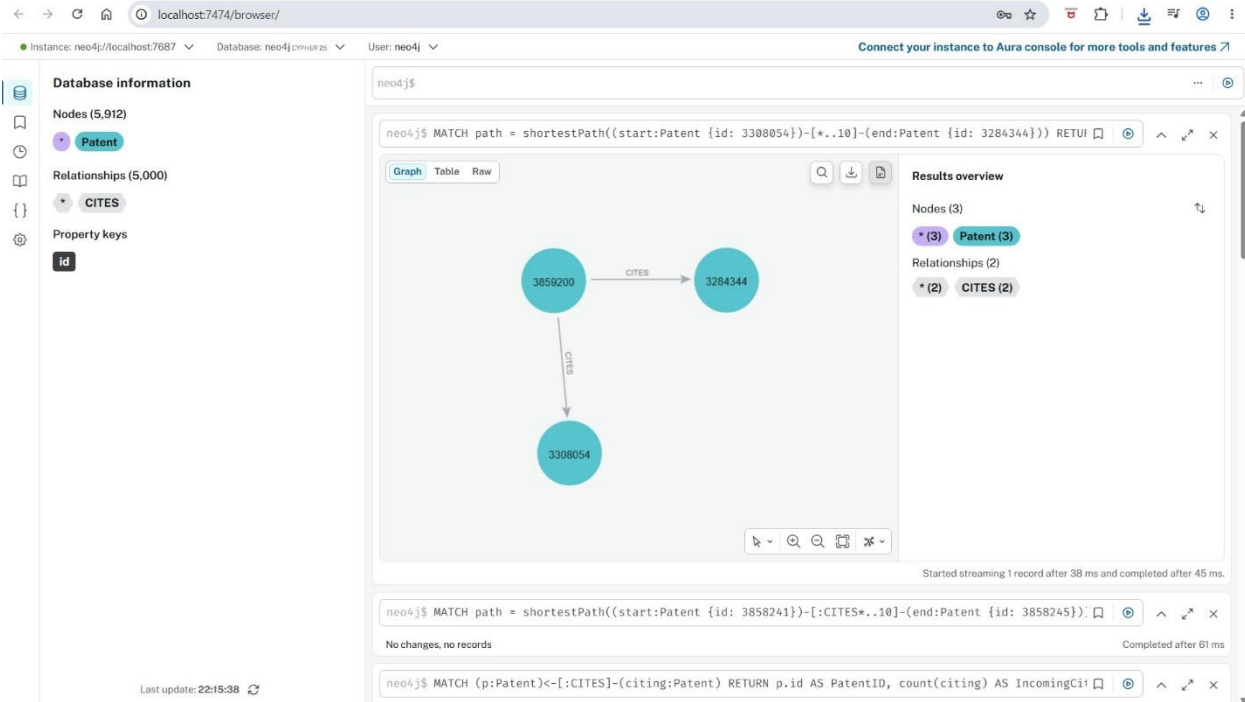
Shortest path analysis evaluates how patents reference prior art across multiple topological jumps. An undirected path traversal was implemented to discover the minimal connection between Patent 3308054 and Patent 3284344.

Cypher Query:

```
MATCH path = shortestPath ((start: Patent {id: '3308054'})- [*..10]-(end: Patent {id: '3284344'}))
```

```
RETURN path
```

- Execution Time: Completed in 45ms (started streaming at 38ms).
- Path findings: the algorithm identified Patent 3859200 as a central connect that directly cities both target patents:
 - (3859200) – [: CITIES] -> (3284344)
 - (3859200) – [: CITIES] -> (3308054)



Screenshot 3: Degree Centrality Results

The Neo4j graph visualization is shown on the screenshot 3 above displaying the 2-hop path between patent 3308054 and 3284344 connected via the central node 3859200.

3.2 Multi-Hop Neighbourhood Traversal

To inspect direct and indirect citations, a query was executed on Patent 3858241 to fetch both 1st degree (direct) and 2nd-degree citations.

To process it Neo4j, this query was added to Neo4j explorer:

MATCH (p: Patent {id: '3858241'})- [r1: CITES]-> (direct: Patent)

OPTIONAL MATCH (direct)- [r2: CITES]-> (second: Patent)

RETURN p.id AS SourcePatent, direct.id AS DirectCitation, collect(second.id) AS SecondDegreeCitations

This took an execution time of around 154 ms returning 5 records.

neo4j\$ MATCH (p:Patent {id: 3858241})-[r1:CITES]->(direct:Patent) OPTIONAL MATCH (direct)-[r2:CITES]->(

Table Raw

	SourcePatent	DirectCitation	SecondDegreeCitations
1	3858241	3634889	[]
2	3858241	3557384	[]
3	3858241	3398406	[]
4	3858241	1324234	[]
5	3858241	956203	[]

Started streaming 5 records after 142 ms and completed after 154 ms.

Screenshot 4: Multi-Hop Neighbourhood results from query 1

Following entry of query 1, the tabular query output displaying 1st degree citations and empty arrays for 2nd degree citations is displayed on screenshot 4.

4. Key Findings and conclusion

- **Direct Out-Degree:** Patent 3858241 cites 5 direct patents in the network, meaning the reference of exactly 6 prior patents within the network.
- **Network Truncation:** All 2nd degree citation lists returned ([]). This emphasizes that the 5 directly cited patents do not have any further outgoing citations link present within this 5000-edge subset of the database.

Task 5: Theoretical Foundations and Future Engineering Trends

Part 5.1: Enterprise Data Governance Framework

When looking to understand enterprise data architecture, there are two types one is known as traditional enterprise data architecture and its counterpart the modern version. It is important to note; traditional enterprise data architecture is dominated by monolithic Data Lakes and centralized Data warehouse which rely on single boundaries of control to implement data quality and governance. While centralized data warehouses provide a strict schema-on-write constraints, strict controls and ACID compliance, subsequently result to organisational bottlenecks as the enterprise structure scale. Furthermore, monolithic data lakes attempt to resolve ingest throughput by basically shifting to schema-on-read but without applying domain-bound accountability, hence unmanaged data lakes tend to fall into ungoverned unstructured data.

Modern distributed architectures resolve much of these bottlenecks through Data Mesh and Data Contracts:

Data Mesh: Decouples data ownership from central engineering teams, casting domain teams as explicit data producers. Moreover, Data is treated as a self-describing product served via standardized platform managed interfaces. It is important to emphasise that Data Mesh architecture is more suited to decentralised data management and allocating access to the relevant domain teams more effectively (IBM,2024).

Data Contract: Enforce declarative, machine-readable specification defining schemas, Data Quality SLAs, freshness metrics and access constraints ensure directly domain boundaries. The fact of the matter is Data contracts are like normal contract, in this case transform data governance from manual, reactive audit into dynamic Governance as code enforced throughout the access usage with integration and ingestion in a continuous manner (IBM, 2024).

To get a better understanding of the difference between Traditional Enterprise data architecture compared to the modern enterprise architecture, please find the table below:

	Traditional Enterprise Data Architecture	Modern Enterprise Data Architecture
Dimension	Centralised Data Warehouses / Monolithic Lakes	Decentralized Data Mesh and Data Contracts
Ownership Model	Centralized IT/ Dara Engineering Team	Decentralized, domain-driven product teams
Schema enforcement	Strict schema-on-write (warehouse)	Contract-first declarative schema validation prior to ingestion

Scaling Dynamics	Sub-linear, central teams face continuous pipeline backlogs	Linear, scale horizontally as new domain teams assuming ownership
Failure Boundaries	Global; upstream operational schema shifts crash downstream workloads	Bounded; breaking changes are blocked at the domain contract boundary
Governance Mode	Manual Policies	Governance-as-code

Regulatory Governness across Big Data Pipelines

When it comes modern regulatory compliance mandates like GDPR and CCPA, the legal framework require automated governance across the entire big data processing lifecycle:

- **Data Minimization and Purpose Limitation:** Data contracts mandate clear fields of access to restrict usage to pre-approved downstream analytics tasks. Ingestion pipelines scrub or isolate non-essential personal identification information (PII) prior to long-term storage (Sogeti Labs, 2025).
- **Right to be Forgotten (Data Erasure):** Regulatory mandates require targeted key data need to be deleted across streaming topics (example log application using Kafka) and immutable, append-only file layout (GDPR, 2016).
- **Dynamic masking:** Ingestion pipelines cryptographically hash or tokenize sensitive attributes at runtime, exposing masked views based on consumer authorisation levels.

Automated Metadate Catalogs and Lineage infrastructure

Active metadata platforms such as Apache Atlas and Open Metadata, operationalize compliance by capturing telemetry from underlying processing engines:

- **Automated Lineage Parsing:** By monitoring query execution hooks (example Spark SQL listeners), catalogs continuously reconstruct Directed Acyclic Graph (DAGs) of data movement, tracking PII propagation across transformation stages.
- **Dynamic Access Policy Enforcement:** Metadata tags attached to assets link directly to security framework like Apache Ranger. This enables engines to dynamically enforce column-level masking and row-level security (RLS) without requiring alternations to underlying to physical storage files.

Part 5.2: Emerging Frontiers- Quantum computing and LLM-driven loop engineering

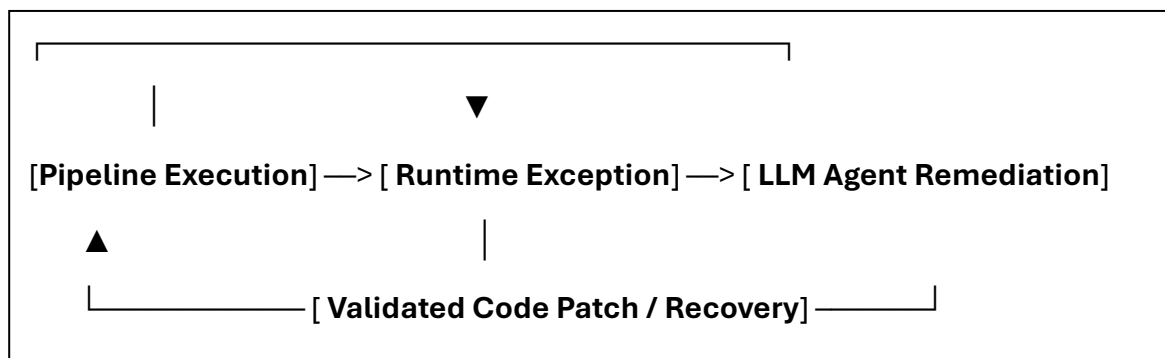
Quantum Big Data Paradigms

Quantum computing replaces binary bits with qubits ($|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$), leveraging superposition and entanglement to fundamentally change processing limits:

1. **NP-Hard Optimisation via Quantum Machine Learning (QML):** QML leverages Parameterized Quantum Circuits (PQCs) and Quantum Support Vector Machines (QSVMs) to compute inner products in high-dimensional Hilbert spaces, which result in skipping classical computational bottleneck for complex optimisation and graph portioning.
2. **Cryptographic signature and security vulnerabilities:** Shor's algorithm solves discrete logarithms and prime factorization in polynomial time ($O((\log N)^3)$), rendering standard asymmetric algorithm vulnerable. This necessitates migrating big data pipelines to Post-Quantum Cryptography (PQC) standards. Symmetric ciphers face quadratic security via Grover's algorithm ($O(\sqrt{N})$), requiring key lengths to double to preserve classical security level (Eureka, 2026).
3. **Multidimensional Indexing and Search:** Grover's algorithm accelerates unstructured data retrieval with quadratic speedups. Quantum linear algebra algorithms solve systems of linear equations in logarithmic time ($O(\log N)$ vs classical $O(N^3)$), speeding up high-dimensional vector index operations for similarity searches (Liu and Moody, 2024).

LLM Loop Engineering

LLM Loop Engineering integrates Large Language Models into closed-loop execution cycles, turning static ETL infrastructure into self-healing, adaptive architectures:



- **Automated DAG & Code Synthesis:** AI agents parse natural language business requirements to synthesize structural Text-to-SQL layer definition, native dbt models and optimized PySpark scripts.
- **Autonomous Self-Healing Pipelines:** When ETL pipelines encounter errors, autonomous agents capture execution logs, evaluate stack traces against historical record, write remedial code patches, validating them in sandbox environments and resume failed jobs.

- **Predictive Pipeline Optimisation:** Autonomous controllers analyse query execution graph to proactively re-index tables, adjust partition strategies and reallocate compute nodes based on anticipated resource usage.

System-Level bottleneck: Nondeterministic LLMs vs Deterministic Runtimes

Integrating non-deterministic LLM output streams into deterministic execution stages (example Apache Spark, Apache Flink) create structural challenges:

- **Execution Graph Instability:** Spark and Flink rely on deterministic, static computational graph (DAGs) to ensure consistent distributed state and data integrity. Non-deterministic LLM-generated logic can introduce unexpected edge cases variations that trigger runtime failures across distributed executor nodes.
- **Latency & Throughput Mismatches:** Micro-batch and real-time streaming engines operate under low-latency SLAs. Moreover, synchronous LLM API calls introduce variable multi-second latencies whilst including severe backpressure across distributed processing technologies.
- **Auditability & Replay Deficits:** Compliance and financial systems mandate reproducible calculation loops; thus ensure the cycle is not at risk of regulatory compliance and system bottlenecks. Non-deterministic transformation logic dynamically modified by LLMs prevents deterministic replay during audit trails and regression testing.

Mitigation: When applying production implantations isolate LLM agents outside core execution engine. The agents propose structural changes through formal validation gates – including static AST parsing, automated unit tests and sandboxed contract validations before being compiled, verified artifacts are deployed through the deterministic processing layers.

Reference List

IBM Think. (2024). What is a Data Mesh? IBM Topics. <https://www.ibm.com/think/topics/data-mesh> (Accessed on 26th August 2026).

IBM Think. (2024). What is a Data Contract? IBM Topics. <https://www.ibm.com/think/topics/data-contract> (Accessed on 26th August 2026).

Ivezic, M. (2024). Post-Quantum Cryptography (PQC) Meets Quantum AI (QAI). Post Quantum. <https://postquantum.com/post-quantum/pqc-quantum-ai-qai/> (Accessed on 27th August 2026).

Sogeti Labs (2025). Emerging Trends: Data Governance in Data Fabrics and Mesh. Sogeti Labs. <https://labs.sogeti.com/emerging-trends-data-governance-in-data-fabrics-and-mesh/> (Accessed on 25th August 2026).

Liu, Y. and Moody, D (2024). Post-quantum cryptography and the quantum future of cybersecurity. National Library of Medicine. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11155471/> (Accessed on 26th August 2026).

Eureka (2026). Post-Quantum Cryptography in Large-Scale Database Security: Key Insights. Eureka by patsnap. <https://eureka.patsnap.com/report-post-quantum-cryptography-in-large-scale-database-security-key-insights> (Accessed 28th August 2026).

GDPR (2016). [General Data Protection Regulation \(GDPR\) – Legal Text](#) (Accessed on 29th August 2026).